

recall_lifecycle

recall_lifecycle.sh — user guide

Revision: 1 **Last modified:** 2026-06-10T12:00:00Z

Overview

tests/e2e/recall_lifecycle.sh is an autonomous, anti-bluff end-to-end challenge that exercises the Helix OTA **server-driven recall (forward-fix rollback) lifecycle** against a live ota-server. It self-hosts the server (with an ephemeral ed25519 signing key on a unique free-probed port), builds genuinely signed OTA artifacts, stages a release v1.1.0 + an all-targets deployment D1, then drives the operator recall to a forward-fix release v1.2.0 and asserts the full contract over real HTTP (curl + jq, no mocks of the system under test).

It closes the recall/forward-fix coverage gap at the e2e/Challenge layer (Constitution §11.4 / §11.4.27 / §11.4.98 / §11.4.123). The matching HelixQA challenge is HOTA-RECALL-LIFECYCLE in tools/helixqa/banks/helix_ota.yaml, which dispatches to this script and scores PASS only on its real exit 0 + captured evidence.

What it asserts (all hard-FAIL on mismatch)

1. POST /api/v1/deployments/{D1}/recall {to_release_id:<v1.2.0>} → **201**, returning a rollback_history row with .kind == "rollback", .from_release_id == <v1.1.0 release>, .to_release_id == <v1.2.0 release>, .details.mode == "forward-fix", a non-empty .recall_deployment_id, and a non-empty .triggered_by.
2. GET /api/v1/deployments/{D1}/rollbacks → **200**, and that same row is present in .items[] (the history is real, not a fire-and-forget 201).
3. GET /api/v1/deployments/{D1} → .status == "superseded" (the recall changed the superseded deployment from active).
4. GET /api/v1/deployments/{recall_deployment_id} → .status == "active" and .release_id == <v1.2.0 release> (the forward-fix is now the active deployment).

Anti-bluff negative controls (no false PASS): a bogus artifact signature → 422 SIGNATURE_INVALID (the server genuinely verifies, so the later signed uploads are real); recall with an empty to_release_id → 400; recall to an absent target release → 404; recall of an absent deployment → 404.

Prerequisites

bash, go, openssl (≥ 3 , with ed25519), xxd, base64, curl, jq, python3. Any missing tool, or an openssl that cannot sign ed25519, yields a SKIP-with-reason (exit 3) — never a false PASS.

Usage examples

```
# Default: self-hosts the server on a free probed port, writes evidence to
```

```
# tests/e2e/RECALL_EVIDENCE.txt.
```

```
bash tests/e2e/recall_lifecycle.sh
```

```
# Pin the port and the evidence file.
```

```
HELIX_RECALL_EVIDENCE=/tmp/recall.txt bash tests/e2e/  
recall_lifecycle.sh --port 8097
```

```
# Reuse a prebuilt server binary (skips the go build).
```

```
bash tests/e2e/recall_lifecycle.sh --server-bin /path/to/ota-  
server
```

Exit codes: 0 = every hard assertion passed (RESULT: PASS); 1 = a hard assertion failed (RESULT: FAIL); 3 = prerequisite/signing SKIP.

Edge cases

- **Unique port.** When `--port/HELIX_PORT` is not given, a free TCP port is probed via python3 so parallel runs never collide.
- **Ephemeral keys.** The ed25519 keypair is generated into a `mktemp` dir and `rm -rf`'d on exit (`trap cleanup EXIT INT TERM`); it is never committed (§11.4.10).
- **Self-cleaning.** The spawned server is killed on every exit path. No host state is touched.
- **Signing unavailable.** If openssl cannot reproduce the server's ed25519 scheme, the upload stage SKIPS-with-reason instead of fabricating a 201.

Internal behaviour

The recall endpoint requires a real, deployable release, which requires a genuinely signed artifact (the server's trust boundary verifies a detached ed25519 signature against its CONFIG-supplied key, never the request's). The script therefore reuses `pipeline_signed.sh`'s exact signing recipe: `sha256_hex` over the whole ZIP, ed25519 sign over the raw 32 digest bytes, base64 of the last 32 DER bytes as the configured pubkey.

Related scripts

- `tests/e2e/pipeline_signed.sh` — the signed artifact → release → deployment → rollout → delta-bearing update-check pipeline this reuses the signing recipe from.
- `tests/e2e/challenge_operational.sh` — the operational + rollout-route E2E.
- `server/internal/api/handlers_recall.go` — `handleRecall / handleListRollbacks` (the system under test).

Last verified date

2026-06-10 — authored; `bash -n + sh -n` clean. Live PASS evidence is captured by the conductor against the clean merged tree (see `docs/qa/<run-id>-recall-challenge/README.md`).